Advanced SQL Practice: Window Functions, Joins & CTEs

Based on Farmer's Market Database

Data Science Practice

Instructions

The following questions are designed to be complex and require deep thinking regarding SQL logic. They cover:

- **Window Functions:** `RANK`, `DENSE_RANK`, `ROW_NUMBER`, `NTILE`, `LEAD`, `LAG`, `FIRST_VALUE`, `SUM() OVER`, `AVG() OVER`.
- **Frames:** `ROWS BETWEEN`, `RANGE BETWEEN`.
- **Joins:** Self Joins, Inner/Left/Right Joins, Cross Joins.
- **CTEs:** Common Table Expressions for breaking down complex logic.

1 Practice Questions

1. **Running Total of Sales:** Calculate the running total of sales (quantity * cost) for each customer, ordered by transaction time. Reset the running total for each customer.
2. **Vendor Rank by Revenue:** Rank all vendors based on their total revenue for each month. Use `DENSE_RANK` to handle ties.
3. **Previous Purchase Price Comparison:** For every transaction, display the current cost per quantity and the cost per quantity of the *previous* transaction for the same product by the same customer.
4. **Next Market Date:** For each market date in the `market_date_info` table, display the next available market date using a window function.
5. **Rolling 7-Day Average:** Calculate the 7-day rolling average of total daily market revenue.
6. **Customer Purchase Gaps:** Identify customers who have gone more than 14 days between two consecutive purchases.
7. **Top 2 Products Per Category:** Find the top 2 selling products (by quantity) for each product category using `ROW_NUMBER`.
8. **Cumulative Revenue Contribution:** Calculate the cumulative percentage of total revenue contributed by each vendor, sorted by their revenue descending. (Pareto analysis prep).
9. **Median Sale Amount:** Find the median transaction total (quantity * cost) for each product category. (Hint: Use `PERCENT_RANK` or `COUNT`).
10. **Inventory Sell-Through Rate:** Calculate the daily sell-through rate (Total Sold / Total Inventory) for each vendor on days they were at the market.
11. **Highest Daily Spender:** Identify the customer who spent the most money on each specific market date.
12. **First vs Last Purchase Price:** For each customer and product, calculate the difference in price paid between their very first purchase and their most recent purchase of that product.
13. **Inter-Transaction Time:** Calculate the average time (in minutes) between transactions for each customer on days where they had multiple transactions.
14. **Price Quartiles:** Categorize all products into 4 price quartiles (Low, Medium-Low, Medium-High, High) based on their average sales price.
15. **Consecutive Days of Sales Increase:** Identify vendors who have had increased revenue for 3 consecutive market days compared to the previous day.
16. **Customer Segmentation (RFM):** Using a CTE, classify customers into 'Gold', 'Silver', and 'Bronze' based on their total spend (Ntile 3).
17. **Booth Hopping Vendors:** Find vendors who have been assigned to at least two different booth numbers in the same month.
18. **Seasoning Sales:** Rank products by total quantity sold within each Season (Spring, Summer, Fall, Winter).

19. **Moving Maximum:** For every transaction, show the maximum quantity sold in any single transaction for that product in the last 10 transactions (Rows Preceding).
20. **Gap Analysis - Empty Market Days:** Find dates between the minimum and maximum market dates where no sales occurred (requires generating a calendar or using Lead/Lag).
21. **Share of Wallet:** Calculate what percentage of a customer's total lifetime spend went to each specific vendor.
22. **Rainy Day Revenue Impact:** Calculate the average daily revenue for days with rain vs. days without rain.
23. **Product Cross-Selling:** Identify pairs of products that were purchased by the same customer on the same day (Self Join).
24. **Exclusive Vendors:** Find customers who have purchased from only one specific vendor throughout their entire history.
25. **Vendor Ranking Fluctuation:** Compare a vendor's revenue rank in the current month vs. their rank in the previous month.
26. **Peak Hours:** Identify the hour of the day (e.g., 09:00, 10:00) that generates the highest revenue for each day of the week.
27. **Slow Moving Inventory:** Identify products that have had inventory listed but zero sales for 3 consecutive market dates.
28. **Month-Over-Month Growth:** Calculate the percentage growth in revenue for each vendor compared to the previous month.
29. **Customer Retention:** Count how many customers from the previous month made a purchase in the current month.
30. **Running Inventory Total:** Calculate the running total of inventory quantity brought by each vendor, resetting every month.
31. **Price Variance:** Find transactions where the cost to customer was more than 10% higher than the average price for that product on that day.
32. **Top Customer by Zip:** Find the top spending customer for each zip code.
33. **Third Purchase Date:** Find the date of the 3rd purchase ever made by every customer. Return NULL if they haven't made 3 purchases.
34. **Vendor Diversity Score:** Rank customers by the number of distinct vendors they have purchased from.
35. **Products Never Sold:** Find products that exist in the product table but have never appeared in the customer_purchases table (Left Join/Not Exists).
36. **Same Day Re-purchase:** Find customers who bought the same product more than once on the same day at different times.
37. **Daily Revenue vs Average:** For each day, calculate the daily total revenue and compare it (difference) to the average daily revenue of the entire dataset.
38. **Dense Rank Price Tiers:** Assign a dense rank to products based on their historical maximum price, grouped by category.

39. **First Time vs Repeat:** For each transaction, flag if it is the customer's first-ever purchase or a repeat purchase.

40. **Vendor-Booth Consistency:** Calculate the percentage of times a vendor is assigned to their "most frequent" booth.

41. **Rolling 3-Market-Day Revenue:** Calculate revenue for the current day plus the next 2 scheduled market days.

42. **Weekend vs Weekday Shoppers:** Classify customers as 'Weekend', 'Weekday', or 'Mixed' based on their purchase days.

43. **Product Popularity Swing:** Identify products that moved from the bottom 25% (quartile 1) in sales quantity in Q1 to the top 25% (quartile 4) in Q2.

44. **Inventory Utilization:** For days where inventory data exists, calculate the ratio of (Quantity Sold / Quantity Available) per vendor.

45. **Time Between Sales:** For a specific vendor (e.g., Vendor ID 7), calculate the time difference between each consecutive sale on a specific day.

46. **Category Revenue Share:** Calculate the monthly revenue of each category and its percentage share of the total monthly revenue.

47. **Missing Inventory Data:** Identify sales transactions that occurred on dates where the vendor has no corresponding record in the `vendor_inventory` table.

48. **Customer Loyalty Streak:** Calculate the longest streak of consecutive market weeks a customer has attended/purchased.

49. **Average Transaction Size Trends:** Calculate the average transaction size (quantity) for the current row's product, considering only the 5 preceding transactions for that product.

50. **The "Whale" Transaction:** Find the single transaction with the highest value for each month, and how much larger it was compared to the month's average transaction.

2 Solutions & SQL Queries

1. Running Total of Sales

```sql
SELECT
    customer_id,
    market_date,
    transaction_time,
    (quantity * cost_to_customer_per_qty) as transaction_value,
    SUM(quantity * cost_to_customer_per_qty) OVER (
        PARTITION BY customer_id
        ORDER BY market_date, transaction_time
    ) as running_total
FROM customer_purchases;
```

2. Vendor Rank by Revenue

```sql
WITH MonthlySales AS (
    SELECT
        vendor_id,
        EXTRACT(YEAR FROM market_date) as yr,
        EXTRACT(MONTH FROM market_date) as mth,
        SUM(quantity * cost_to_customer_per_qty) as monthly_revenue
    FROM customer_purchases
    GROUP BY vendor_id, yr, mth
)
SELECT
    vendor_id, yr, mth, monthly_revenue,
    DENSE_RANK() OVER (PARTITION BY yr, mth ORDER BY monthly_revenue DESC) as
    rank_in_month
FROM MonthlySales;
```

3. Previous Purchase Price Comparison

```sql
SELECT
    customer_id,
    product_id,
    market_date,
    cost_to_customer_per_qty as current_price,
    LAG(cost_to_customer_per_qty, 1) OVER (
        PARTITION BY customer_id, product_id
        ORDER BY market_date, transaction_time
    ) as prev_transaction_price
FROM customer_purchases;
```

4. Next Market Date

```sql
SELECT
    market_date,
    market_day,
    LEAD(market_date, 1) OVER (ORDER BY market_date) as next_market_date
FROM market_date_info;
```

5. Rolling 7-Day Average

```sql
WITH DailySales AS (
    SELECT market_date, SUM(quantity * cost_to_customer_per_qty) as daily_total
    FROM customer_purchases
    GROUP BY market_date
)
SELECT
    market_date,
    daily_total,
    AVG(daily_total) OVER (
        ORDER BY market_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) as rolling_7_day_avg
FROM DailySales;
```

6. Customer Purchase Gaps

```sql
WITH PurchaseDates AS (
    SELECT DISTINCT customer_id, market_date
    FROM customer_purchases
),
GapCalculation AS (
    SELECT
        customer_id,
        market_date,
        LAG(market_date) OVER (PARTITION BY customer_id ORDER BY market_date) as prev_date,
        DATEDIFF(market_date, LAG(market_date) OVER (PARTITION BY customer_id ORDER BY market_date)) as days_diff
    FROM PurchaseDates
)
SELECT * FROM GapCalculation WHERE days_diff > 14;
```

7. Top 2 Products Per Category

```sql
WITH ProductSales AS (
    SELECT
        p.product_category_id,
        cp.product_id,
        SUM(cp.quantity) as total_qty
    FROM customer_purchases cp
    JOIN product p ON cp.product_id = p.product_id
    GROUP BY p.product_category_id, cp.product_id
),
RankedProducts AS (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY product_category_id ORDER BY total_qty DESC) as rn
    FROM ProductSales
)
SELECT * FROM RankedProducts WHERE rn <= 2;
```

8. Cumulative Revenue Contribution

```sql
WITH VendorTotal AS (
    SELECT vendor_id, SUM(quantity * cost_to_customer_per_qty) as total_rev
    FROM customer_purchases
    GROUP BY vendor_id
),
TotalMarket AS (
    SELECT SUM(total_rev) as grand_total FROM VendorTotal
)
SELECT
    v.vendor_id,
    v.total_rev,
    SUM(v.total_rev) OVER (ORDER BY v.total_rev DESC) as running_rev,
    (SUM(v.total_rev) OVER (ORDER BY v.total_rev DESC) / t.grand_total) * 100 as
    cumulative_pct
FROM VendorTotal v
CROSS JOIN TotalMarket t;
```

9. Median Sale Amount

```sql
SELECT DISTINCT
    product_category_id,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY (quantity * cost_to_customer_per_qty))
    OVER (PARTITION BY product_category_id) as median_sale
FROM customer_purchases cp
JOIN product p ON cp.product_id = p.product_id;
```

10. Inventory Sell-Through Rate

```sql
WITH Sales AS (
    SELECT vendor_id, market_date, product_id, SUM(quantity) as sold_qty
    FROM customer_purchases
    GROUP BY vendor_id, market_date, product_id
)
SELECT
    s.market_date,
    s.vendor_id,
    s.product_id,
    s.sold_qty,
    vi.quantity as inventory_qty,
    (s.sold_qty / vi.quantity) * 100 as sell_through_rate
FROM Sales s
JOIN vendor_inventory vi
    ON s.vendor_id = vi.vendor_id
    AND s.product_id = vi.product_id
    AND s.market_date = vi.market_date;
```

11. Highest Daily Spender

```sql
WITH DailySpend AS (
    SELECT
        market_date,
        customer_id,
        SUM(quantity * cost_to_customer_per_qty) as spend
    FROM customer_purchases
    GROUP BY market_date, customer_id
),
RankedSpend AS (
    SELECT *, RANK() OVER (PARTITION BY market_date ORDER BY spend DESC) as rn
```

```
    FROM DailySpend
)
SELECT * FROM RankedSpend WHERE rn = 1;
```

12. First vs Last Purchase Price

```
SELECT DISTINCT
    customer_id,
    product_id,
    FIRST_VALUE(cost_to_customer_per_qty) OVER (PARTITION BY customer_id, product_id
    ORDER BY market_date, transaction_time ASC) as first_price,
    FIRST_VALUE(cost_to_customer_per_qty) OVER (PARTITION BY customer_id, product_id
    ORDER BY market_date, transaction_time DESC) as last_price
FROM customer_purchases;
```

13. Inter-Transaction Time

```
WITH TimeDiffs AS (
    SELECT
        customer_id,
        market_date,
        transaction_time,
        LAG(transaction_time) OVER (PARTITION BY customer_id, market_date ORDER BY
    transaction_time) as prev_time
    FROM customer_purchases
)
SELECT
    customer_id,
    AVG(TIMESTAMPDIFF(MINUTE, prev_time, transaction_time)) as avg_minutes_between
FROM TimeDiffs
WHERE prev_time IS NOT NULL
GROUP BY customer_id;
```

14. Price Quartiles

```
WITH AvgPrices AS (
    SELECT product_id, AVG(cost_to_customer_per_qty) as avg_price
    FROM customer_purchases
    GROUP BY product_id
)
SELECT
    product_id,
    avg_price,
    NTILE(4) OVER (ORDER BY avg_price) as price_quartile
FROM AvgPrices;
```

15. Consecutive Days of Sales Increase

```
WITH DailyRev AS (
    SELECT vendor_id, market_date, SUM(quantity*cost_to_customer_per_qty) as rev
    FROM customer_purchases
    GROUP BY vendor_id, market_date
),
Growth AS (
    SELECT
        vendor_id,
```

```
        market_date,
        rev,
        LAG(rev) OVER (PARTITION BY vendor_id ORDER BY market_date) as prev_rev
    FROM DailyRev
),
Flags AS (
    SELECT *, CASE WHEN rev > prev_rev THEN 1 ELSE 0 END as is_increase
    FROM Growth
)
SELECT vendor_id, market_date
FROM (
    SELECT
        *,
        SUM(is_increase) OVER (PARTITION BY vendor_id ORDER BY market_date ROWS BETWEEN 2
     PRECEDING AND CURRENT ROW) as increase_streak
    FROM Flags
) t
WHERE increase_streak = 3;
```

16. Customer Segmentation (RFM)

```
WITH CustomerTotal AS (
    SELECT customer_id, SUM(quantity * cost_to_customer_per_qty) as total_spend
    FROM customer_purchases
    GROUP BY customer_id
)
SELECT
    customer_id,
    total_spend,
    NTILE(3) OVER (ORDER BY total_spend DESC) as tier
    -- 1=Gold, 2=Silver, 3=Bronze
FROM CustomerTotal;
```

17. Booth Hopping Vendors

```
SELECT
    v.vendor_name,
    EXTRACT(YEAR_MONTH FROM vba.market_date) as mth,
    COUNT(DISTINCT vba.booth_number) as distinct_booths
FROM vendor_booth_assignments vba
JOIN vendor v ON vba.vendor_id = v.vendor_id
GROUP BY v.vendor_name, mth
HAVING distinct_booths >= 2;
```

18. Seasoning Sales

```
SELECT
    mdi.market_season,
    p.product_name,
    SUM(cp.quantity) as seasonal_qty,
    RANK() OVER (PARTITION BY mdi.market_season ORDER BY SUM(cp.quantity) DESC) as
    rank_in_season
FROM customer_purchases cp
JOIN market_date_info mdi ON cp.market_date = mdi.market_date
JOIN product p ON cp.product_id = p.product_id
GROUP BY mdi.market_season, p.product_name;
```

19. Moving Maximum

```sql
SELECT
    transaction_time,
    product_id,
    quantity,
    MAX(quantity) OVER (
        PARTITION BY product_id
        ORDER BY market_date, transaction_time
        ROWS BETWEEN 10 PRECEDING AND 1 PRECEDING
    ) as max_qty_last_10
FROM customer_purchases;
```

20. Gap Analysis - Empty Market Days

```sql
SELECT mdi.market_date
FROM market_date_info mdi
LEFT JOIN customer_purchases cp ON mdi.market_date = cp.market_date
WHERE cp.market_date IS NULL;
```

21. Share of Wallet

```sql
WITH CustVendorSpend AS (
    SELECT customer_id, vendor_id, SUM(quantity * cost_to_customer_per_qty) as spend
    FROM customer_purchases
    GROUP BY customer_id, vendor_id
)
SELECT
    customer_id,
    vendor_id,
    spend,
    SUM(spend) OVER (PARTITION BY customer_id) as total_cust_spend,
    (spend / SUM(spend) OVER (PARTITION BY customer_id)) * 100 as share_of_wallet
FROM CustVendorSpend;
```

22. Rainy Day Revenue Impact

```sql
SELECT
    CASE WHEN mdi.market_rain_flag = 1 THEN 'Rain' ELSE 'No Rain' END as weather,
    AVG(daily_sales) as avg_revenue
FROM (
    SELECT market_date, SUM(quantity * cost_to_customer_per_qty) as daily_sales
    FROM customer_purchases
    GROUP BY market_date
) sales
JOIN market_date_info mdi ON sales.market_date = mdi.market_date
GROUP BY mdi.market_rain_flag;
```

23. Product Cross-Selling

```sql
SELECT
    t1.customer_id,
    t1.market_date,
    t1.product_id as prod_A,
    t2.product_id as prod_B
FROM customer_purchases t1
```

```sql
JOIN customer_purchases t2
    ON t1.customer_id = t2.customer_id
    AND t1.market_date = t2.market_date
    AND t1.product_id < t2.product_id; -- Avoid duplicates and self-matches
```

24. Exclusive Vendors

```sql
WITH VendorCounts AS (
    SELECT customer_id, COUNT(DISTINCT vendor_id) as vendor_count, MAX(vendor_id) as
    sole_vendor
    FROM customer_purchases
    GROUP BY customer_id
)
SELECT c.customer_first_name, c.customer_last_name, v.vendor_name
FROM VendorCounts vc
JOIN customer c ON vc.customer_id = c.customer_id
JOIN vendor v ON vc.sole_vendor = v.vendor_id
WHERE vc.vendor_count = 1;
```

25. Vendor Ranking Fluctuation

```sql
WITH MonthlyRev AS (
    SELECT
        vendor_id,
        EXTRACT(MONTH FROM market_date) as mth,
        SUM(quantity*cost_to_customer_per_qty) as rev
    FROM customer_purchases
    GROUP BY vendor_id, mth
),
Ranked AS (
    SELECT *, RANK() OVER (PARTITION BY mth ORDER BY rev DESC) as rn
    FROM MonthlyRev
)
SELECT
    r1.vendor_id,
    r1.mth as current_month,
    r1.rn as current_rank,
    r2.rn as prev_rank,
    (r2.rn - r1.rn) as rank_improvement
FROM Ranked r1
LEFT JOIN Ranked r2 ON r1.vendor_id = r2.vendor_id AND r1.mth = r2.mth + 1;
```

26. Peak Hours

```sql
SELECT
    market_day,
    EXTRACT(HOUR FROM transaction_time) as hour_of_day,
    SUM(quantity * cost_to_customer_per_qty) as total_rev
FROM customer_purchases cp
JOIN market_date_info mdi ON cp.market_date = mdi.market_date
GROUP BY market_day, hour_of_day
ORDER BY market_day, total_rev DESC;
```

27. Slow Moving Inventory

```sql
SELECT DISTINCT i.product_id, i.vendor_id
FROM vendor_inventory i
LEFT JOIN customer_purchases s
    ON i.product_id = s.product_id
    AND i.vendor_id = s.vendor_id
    AND i.market_date = s.market_date
WHERE s.product_id IS NULL; -- Logic assumes 'Slow Moving' means listed but not sold on
    that day
```

28. Month-Over-Month Growth

```sql
WITH MonthlyStats AS (
    SELECT
        vendor_id,
        EXTRACT(YEAR_MONTH FROM market_date) as ym,
        SUM(quantity * cost_to_customer_per_qty) as revenue
    FROM customer_purchases
    GROUP BY vendor_id, ym
)
SELECT
    vendor_id,
    ym,
    revenue,
    LAG(revenue) OVER (PARTITION BY vendor_id ORDER BY ym) as prev_revenue,
    ((revenue - LAG(revenue) OVER (PARTITION BY vendor_id ORDER BY ym)) / LAG(revenue)
    OVER (PARTITION BY vendor_id ORDER BY ym)) * 100 as growth_pct
FROM MonthlyStats;
```

29. Customer Retention

```sql
WITH Buyers AS (
    SELECT DISTINCT customer_id, EXTRACT(MONTH FROM market_date) as mth
    FROM customer_purchases
)
SELECT
    b1.mth as current_month,
    COUNT(b1.customer_id) as retained_customers
FROM Buyers b1
JOIN Buyers b2 ON b1.customer_id = b2.customer_id AND b1.mth = b2.mth + 1
GROUP BY b1.mth;
```

30. Running Inventory Total

```sql
SELECT
    vendor_id,
    market_date,
    quantity,
    SUM(quantity) OVER (
        PARTITION BY vendor_id, EXTRACT(YEAR_MONTH FROM market_date)
        ORDER BY market_date
    ) as mtd_inventory_brought
FROM vendor_inventory;
```

31. Price Variance

```sql
WITH AvgDailyPrice AS (
    SELECT product_id, market_date, AVG(cost_to_customer_per_qty) as daily_avg_cost
    FROM customer_purchases
    GROUP BY product_id, market_date
)
SELECT cp.*
FROM customer_purchases cp
JOIN AvgDailyPrice adp
    ON cp.product_id = adp.product_id
    AND cp.market_date = adp.market_date
WHERE cp.cost_to_customer_per_qty > (adp.daily_avg_cost * 1.10);
```

32. Top Customer by Zip

```sql
WITH ZipSpend AS (
    SELECT
        c.customer_zip,
        c.customer_id,
        SUM(cp.quantity * cp.cost_to_customer_per_qty) as total_spend
    FROM customer c
    JOIN customer_purchases cp ON c.customer_id = cp.customer_id
    GROUP BY c.customer_zip, c.customer_id
),
Ranked AS (
    SELECT *, RANK() OVER (PARTITION BY customer_zip ORDER BY total_spend DESC) as rn
    FROM ZipSpend
)
SELECT * FROM Ranked WHERE rn = 1;
```

33. Third Purchase Date

```sql
WITH OrdinalPurchases AS (
    SELECT
        customer_id,
        market_date,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY market_date,
    transaction_time) as purchase_num
    FROM customer_purchases
)
SELECT customer_id, market_date
FROM OrdinalPurchases
WHERE purchase_num = 3;
```

34. Vendor Diversity Score

```sql
SELECT
    customer_id,
    COUNT(DISTINCT vendor_id) as distinct_vendors
FROM customer_purchases
GROUP BY customer_id
ORDER BY distinct_vendors DESC;
```

35. Products Never Sold

```sql
SELECT p.product_name
FROM product p
LEFT JOIN customer_purchases cp ON p.product_id = cp.product_id
WHERE cp.product_id IS NULL;
```

36. Same Day Re-purchase

```sql
SELECT
    customer_id,
    product_id,
    market_date,
    COUNT(*) as purchase_count
FROM customer_purchases
GROUP BY customer_id, product_id, market_date
HAVING COUNT(*) > 1;
```

37. Daily Revenue vs Average

```sql
WITH Daily AS (
    SELECT market_date, SUM(quantity*cost_to_customer_per_qty) as rev
    FROM customer_purchases
    GROUP BY market_date
),
OverallAvg AS (
    SELECT AVG(rev) as global_avg FROM Daily
)
SELECT
    d.market_date,
    d.rev,
    (d.rev - oa.global_avg) as diff_from_avg
FROM Daily d
CROSS JOIN OverallAvg oa;
```

38. Dense Rank Price Tiers

```sql
WITH MaxPrices AS (
    SELECT product_id, MAX(cost_to_customer_per_qty) as max_price
    FROM customer_purchases
    GROUP BY product_id
)
SELECT
    p.product_category_id,
    mp.product_id,
    mp.max_price,
    DENSE_RANK() OVER (PARTITION BY p.product_category_id ORDER BY mp.max_price DESC) as
    price_rank
FROM MaxPrices mp
JOIN product p ON mp.product_id = p.product_id;
```

39. First Time vs Repeat

```sql
SELECT
    customer_id,
    market_date,
    CASE
```

```
        WHEN market_date = MIN(market_date) OVER (PARTITION BY customer_id) THEN 'First Time'
        ELSE 'Repeat'
    END as purchase_type
FROM customer_purchases;
```

40. Vendor-Booth Consistency

```
WITH BoothCounts AS (
    SELECT vendor_id, booth_number, COUNT(*) as times_assigned
    FROM vendor_booth_assignments
    GROUP BY vendor_id, booth_number
),
MaxBooth AS (
    SELECT vendor_id, MAX(times_assigned) as max_count FROM BoothCounts GROUP BY vendor_id
),
TotalAssignments AS (
    SELECT vendor_id, COUNT(*) as total FROM vendor_booth_assignments GROUP BY vendor_id
)
SELECT
    mb.vendor_id,
    (mb.max_count / ta.total) * 100 as consistency_score
FROM MaxBooth mb
JOIN TotalAssignments ta ON mb.vendor_id = ta.vendor_id;
```

41. Rolling 3-Market-Day Revenue

```
WITH DailySales AS (
    SELECT market_date, SUM(quantity * cost_to_customer_per_qty) as sales
    FROM customer_purchases
    GROUP BY market_date
)
SELECT
    market_date,
    sales,
    SUM(sales) OVER (
        ORDER BY market_date
        ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING
    ) as revenue_next_3_days
FROM DailySales;
```

42. Weekend vs Weekday Shoppers

```
SELECT
    cp.customer_id,
    CASE
        WHEN COUNT(DISTINCT CASE WHEN mdi.market_day IN ('Saturday', 'Sunday') THEN cp.market_date END) = COUNT(DISTINCT cp.market_date) THEN 'Weekend Only'
        WHEN COUNT(DISTINCT CASE WHEN mdi.market_day NOT IN ('Saturday', 'Sunday') THEN cp.market_date END) = COUNT(DISTINCT cp.market_date) THEN 'Weekday Only'
        ELSE 'Mixed'
    END as shopper_type
FROM customer_purchases cp
JOIN market_date_info mdi ON cp.market_date = mdi.market_date
GROUP BY cp.customer_id;
```

43. Product Popularity Swing

```sql
-- Requires dividing data into quarters (Q1, Q2) based on date, then calculating quartiles of sales quantity, then finding products switching buckets.
-- Simplified version:
WITH QSales AS (
    SELECT
        product_id,
        CASE WHEN EXTRACT(MONTH FROM market_date) BETWEEN 1 AND 3 THEN 'Q1' ELSE 'Q2' END as qtr,
        SUM(quantity) as qty
    FROM customer_purchases
    GROUP BY product_id, 2
),
Ranked AS (
    SELECT *, NTILE(4) OVER (PARTITION BY qtr ORDER BY qty) as bucket FROM QSales
)
SELECT r1.product_id
FROM Ranked r1
JOIN Ranked r2 ON r1.product_id = r2.product_id
WHERE r1.qtr = 'Q1' AND r1.bucket = 1 AND r2.qtr = 'Q2' AND r2.bucket = 4;
```

44. Inventory Utilization

```sql
WITH Sold AS (
    SELECT vendor_id, product_id, market_date, SUM(quantity) as sold
    FROM customer_purchases GROUP BY 1,2,3
)
SELECT
    s.vendor_id,
    s.market_date,
    (s.sold / vi.quantity) as utilization
FROM Sold s
JOIN vendor_inventory vi
    ON s.vendor_id=vi.vendor_id AND s.product_id=vi.product_id AND s.market_date=vi.market_date
WHERE vi.quantity > 0;
```

45. Time Between Sales

```sql
SELECT
    transaction_time,
    transaction_time - LAG(transaction_time) OVER (ORDER BY transaction_time) as time_gap
FROM customer_purchases
WHERE vendor_id = 7 AND market_date = '2019-04-03';
```

46. Category Revenue Share

```sql
WITH CatMonthly AS (
    SELECT
        p.product_category_id,
        EXTRACT(YEAR_MONTH FROM cp.market_date) as ym,
        SUM(cp.quantity * cp.cost_to_customer_per_qty) as rev
    FROM customer_purchases cp
    JOIN product p ON cp.product_id = p.product_id
    GROUP BY 1,2
)
SELECT
```

```
    product_category_id,
    ym,
    rev,
    rev / SUM(rev) OVER (PARTITION BY ym) as monthly_share
FROM CatMonthly;
```

47. Missing Inventory Data

```
SELECT DISTINCT cp.vendor_id, cp.product_id, cp.market_date
FROM customer_purchases cp
LEFT JOIN vendor_inventory vi
    ON cp.vendor_id=vi.vendor_id AND cp.product_id=vi.product_id AND cp.market_date=vi.
    market_date
WHERE vi.vendor_id IS NULL;
```

48. Customer Loyalty Streak

```
-- Complex: Requires mapping weeks, finding gaps, and calculating max consecutive
    sequence.
WITH Weeks AS (
    SELECT DISTINCT customer_id, EXTRACT(WEEK FROM market_date) as wk, EXTRACT(YEAR FROM
    market_date) as yr
    FROM customer_purchases
),
Grouped AS (
    SELECT
        customer_id,
        wk,
        wk - ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY yr, wk) as grp
    FROM Weeks
)
SELECT customer_id, COUNT(*) as streak_length
FROM Grouped
GROUP BY customer_id, grp
ORDER BY streak_length DESC LIMIT 1;
```

49. Average Transaction Size Trends

```
SELECT
    product_id,
    market_date,
    quantity,
    AVG(quantity) OVER (
        PARTITION BY product_id
        ORDER BY market_date, transaction_time
        ROWS BETWEEN 5 PRECEDING AND 1 PRECEDING
    ) as avg_prev_5
FROM customer_purchases;
```

50. The ”Whale” Transaction

```
WITH MonthlyAvg AS (
    SELECT
        EXTRACT(YEAR_MONTH FROM market_date) as ym,
        AVG(quantity * cost_to_customer_per_qty) as avg_val
    FROM customer_purchases
```

```
    GROUP BY ym
),
Transactions AS (
    SELECT
        transaction_time,
        EXTRACT(YEAR_MONTH FROM market_date) as ym,
        (quantity * cost_to_customer_per_qty) as val
    FROM customer_purchases
)
SELECT
    t.ym,
    MAX(t.val) as max_transaction,
    (MAX(t.val) / ma.avg_val) as multiple_of_average
FROM Transactions t
JOIN MonthlyAvg ma ON t.ym = ma.ym
GROUP BY t.ym, ma.avg_val;
```